
Spring Security

>> What is Spring Security ?

-- Spring Security is a framework that secures applications :

1. Authentication – Who the user is.
2. Authorization – What access the user has.
3. CSRF / CORS – Protects against cross-site attacks and controls resource sharing.
4. JWT / OAuth2 – Token-based and delegated authentication.
5. Session Management – Manages user sessions securely.
6. Password Encryption – Safely stores and verifies passwords.

>> Authentication vs Authorization

==========

>> Authentication

The user logs in and gets verified.
Example : username / password

==========

>> Authorization

After login , the user gets access based on roles / permissions.
Example : ADMIN can delete, USER cannot.

==========

>> Spring Security Flow

Request comes → Security Filter Chain → Authentication → Authorization → Controller

Spring Security works entirely on filters.

>> Security Filters

-- In Spring Security, there is a chain of filters :

1. UsernamePasswordAuthenticationFilter
2. BasicAuthenticationFilter
3. JwtAuthenticationFilter (custom)
4. ExceptionTranslationFilter
5. AuthorizationFilter

>> Default Spring Security Behavior

If we add the dependency : spring-boot-starter-security

Then :

1. Every endpoint becomes secure
2. Default login page appears
3. Default username : user
4. Password is printed in the console

==========

>> Modern Spring Security Config

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

==========

1. @Configuration : Tells Spring: this class contains configuration.

2. @EnableWebSecurity : Enables Spring Security for the project.

3. SecurityFilterChain (Main Security Setup)
-- This defines how security should work for all requests.

4. CSRF Disabled : .csrf(csrf -> csrf.disable())
-- CSRF is mostly needed for form login + session apps
-- Since you are using HTTP Basic (mostly API use), it is disabled.

5. Authorization Rules : .authorizeHttpRequests(auth -> auth
-- This block decides who can access what.

>> Public endpoints : .requestMatchers("/public/**").permitAll()

-- Anyone can access /public/...
-- No login needed

>> Admin endpoints : .requestMatchers("/admin/**").hasRole("ADMIN")

-- Only users having role ADMIN can access /admin/...
-- Spring internally checks for ROLE_ADMIN

```
>>  All other endpoints  :  .anyRequest().authenticated()
--  Any other URL requires login

6. Login Type = HTTP Basic  :  .httpBasic(Customizer.withDefaults());

--  Enables HTTP Basic Authentication
--  Works easily in Postman
--  Browser shows popup login



--------------------------------------------------------------------------------
--

>>  Roles Vs Authorities

===========

>>  Role

--  Spring internally adds the prefix "ROLE_" to roles.

Example : hasRole("ADMIN") means "ROLE_ADMIN"

===========

>>  Authority

Direct permission (no prefix).

Examples : "READ_PRIVILEGE" | "WRITE_PRIVILEGE"

===========

>>  In Memory Users (Testing)


@Bean
public UserDetailsService userDetailsService() {

    UserDetails user = User.withUsername("user")
            .password(passwordEncoder().encode("1234"))
            .roles("USER")
            .build();

    UserDetails admin = User.withUsername("admin")
            .password(passwordEncoder().encode("admin123"))
            .roles("ADMIN")
            .build();

    return new InMemoryUserDetailsManager(user, admin);
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

===========

This code is creating in-memory users (not from DB) for Spring Security.

1.  userDetailsService()  :  @Bean public UserDetailsService
userDetailsService()
```

-- This tells Spring Security : Use these users for authentication.

2. Creating USER

```
UserDetails user = User.withUsername("user")
        .password(passwordEncoder().encode("1234"))
        .roles("USER")
        .build();
```

-- Username : user
-- Password : 1234 (stored as BCrypt encoded)
-- Role : USER (internally becomes ROLE_USER)

3. Creating ADMIN

```
UserDetails admin = User.withUsername("admin")
        .password(passwordEncoder().encode("admin123"))
        .roles("ADMIN")
        .build();
```

-- Username : admin
-- Password : admin123
-- Role : ADMIN (internally becomes ROLE_ADMIN)

4. Storing users in memory : return new InMemoryUserDetailsManager(user, admin);

-- This stores both users inside application memory.
-- So Spring Security will authenticate using these users.

5. Password Encoder

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

>> Spring does not allow plain passwords, so this encoder is used to :
-- encrypt password during storage , match password during login

---
--

>> Database Authentication (Important For Projects)

Table Structure

users : id | username | password |enabled

roles : id | name

user_roles : user_id | role_id

==========

>> User Entity Code

```
@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
    private String password;
    private boolean enabled = true;

    @ManyToMany(fetch = FetchType.EAGER)
    private Set<Role> roles = new HashSet<>();
}

===========

>>  @Entity : Means this class will become a table in DB (ex: user or users).

>>  Primary Key : id is the primary key , Auto-increment in DB

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;


>>  User Fields

--  username → login username
--  password → stored in encrypted form (BCrypt)
--  enabled → if false , user cannot login

private String username;
private String password;
private boolean enabled = true;


>>  Roles Mapping (Many-to-Many)

@ManyToMany(fetch = FetchType.EAGER)
private Set<Role> roles = new HashSet<>();


--  One user can have multiple roles
--  One role can belong to multiple users
--  EAGER means : when user loads, roles also load immediately
--  Stored using a join table (like user_roles)

-------------------------------------------------------------------------------
--

>>  UserDetails and UserDetailsService

===========

>>  UserDetails
--  Spring needs user data in this format.


>>  UserDetailsService
--  Fetches the user from the database.

===========
```

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("User not
found"));

        return org.springframework.security.core.userdetails.User
                .withUsername(user.getUsername())
                .password(user.getPassword())
                .authorities(user.getRoles().stream()
                        .map(r -> "ROLE_" + r.getName())
                        .toArray(String[]::new))
                .build();
    }
}

===========

1.  @Service : Spring will create this as a bean automatically.

2.  Implements UserDetailsService : Spring Security calls this method during
login : loadUserByUsername(username)

3.  Fetch user from DB

--  Finds user by username from database
--  If not found → throws exception → login fails

4.  Convert DB User → Spring Security UserDetails

--  Spring Security needs user data in UserDetails format.
--  username taken from DB
--  password taken from DB (already BCrypt encrypted)

5.  Set roles as authorities

--  Converts roles into authorities
--  Adds "ROLE_" prefix manually

Example : If role name = ADMIN → becomes ROLE_ADMIN

-------------------------------------------------------------------------------
--

>>  Password Encoding

--  Spring does not allow plain-text passwords.

Best encoder : BCryptPasswordEncoder

-------------------------------------------------------------------------------
--

>>  Login Types

1.  HTTP Basic Auth
--  Easy in Postman | Browser shows a popup
```

2. Form Login
-- Spring default login page or a custom page

3. JWT Token Login
-- Most used in REST APIs

==========

>> What is CSRF ? : Cross-Site Request Forgery

-- An attack where a logged-in user is tricked into performing an unwanted action.

When is it enabled ? : Form Login + Session-based applications

In REST APIs ? : Usually disabled

```
http.csrf(csrf -> csrf.disable());
```

==========

>> CORS (Must Know For FrontEnd)

-- When a request comes from a frontend like Angular, we may face CORS issues because the browser blocks cross-origin requests.

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:3000"));
    config.setAllowedMethods(List.of("GET","POST","PUT","DELETE"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

==========

>> It allow requests from : http://localhost:3000 (Angular) to our Spring backend.

1. Create CORS config : CorsConfiguration config = new CorsConfiguration();

2. Allowed Origins : config.setAllowedOrigins(List.of("http://localhost:3000"));

-- Only React running on localhost:3000 can access the backend.

3. Allowed Methods : Only these HTTP methods are allowed.

```
config.setAllowedMethods(List.of("GET","POST","PUT","DELETE"));
```

4. Allowed Headers : config.setAllowedHeaders(List.of("*"));

-- Allows all headers like : Authorization , Content-Type , etc.

5. Allow Credentials : config.setAllowCredentials(true);

-- Allows cookies / session / authorization headers to be sent.

6. Apply to all endpoints : CORS config will apply to every API URL.

source.registerCorsConfiguration("/**", config);

==========

Now your frontend can call your backend like : without getting CORS error.

http://localhost:3000 → http://localhost:8080/api/...
